



OBJETIVO

Optimizar el rendimiento de la base de datos GradosTítulos mediante el monitoreo del comportamiento del motor SQL Server, análisis de consultas, índices, transacciones y administración de recursos para garantizar tiempos de respuesta adecuados en los procesos de grados y títulos.



TEMAS PRINCIPALES

- ✓ Uso de SQL Profiler y Extended Events.
- ✓ Estadísticas e índices (creación, fragmentación, mantenimiento).
- ✓ Administración de transacciones y bloqueos.
- ✓ Análisis de planes de ejecución.
- ✓ Optimización de consultas T-SQL.
- ✓ Control de recursos con Resource Governor.



ACTIVIDAD

- 1 Realiza 6 infografías con los temas de la sesión de aprendizaje.
- 2 Responde con claridad y precisión cada enunciado propuesto.
- 3 Formato de cada caso:
 - Enunciado del ejercicio
 - Script de la solución en T-SQL
 - Justificación técnica
 - Explicación de buenas prácticas

1 PROYECTO 1: RESPALDO COMPLETO INSTITUCIONAL DE LA BASE DE DATOS GRADOSTITULOS



ENUNCIADO

La Oficina de Tecnologías de Información ha detectado lentitud durante el registro de expedientes de bachiller y título profesional presenta lentitud durante las horas de mayor demanda.

Se solicita implementar una sesión de monitoreo utilizando Extended Events para capturar todas las consultas cuya duración sea superior a 1000 milisegundos sobre la base de datos GradosTítulos.

El estudiante deberá identificar las consultas más costosas ejecutadas sobre los esquemas:

- Académico
- Trámite
- Evaluación
- Documento

Al finalizar, deberá generar un reporte con las consultas capturadas.

SOLUCIÓN (T-SQL)

```
-- Crear la sesión de Extended Events
CREATE EVENT SESSION MonitoreoConsultas
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
  ACTION
  (
    sqlserver.database_name,
    sqlserver.sql_text
  )
  WHERE duration > 1000000 -- 1000 ms
  AND database_name = 'GradosTítulos'
)
ADD TARGET package0.event_file
(
  SET filename = 'C:\Eventos\ConsultasLentas.xel',
  max_file_size = 50,
  max_rollover_files = 10
);

ALTER EVENT SESSION MonitoreoConsultas
ON SERVER STATE = START;

Para generar el reporte:
SELECT
DATEADD(ms, duration/1000.0, '1970-01-01') AS [Fecha],
duration/1000.0 AS Duracion_ms,
database_name,
sql_text
FROM sys.fn_xe_file_target_read_file(
'C:\Eventos\ConsultasLentas.xel', NULL, NULL, NULL)
ORDER BY Duracion_ms DESC;
```



JUSTIFICACIÓN TÉCNICA

Extended Events tiene menor impacto en el rendimiento que SQL Profiler y permite filtrar por duración, capturando únicamente las consultas lentas.

Se define el umbral en 1000 ms y se almacena en archivo para su posterior análisis y generación de reportes.



BUENAS PRÁCTICAS

- Capturar solo los eventos necesarios para reducir el impacto en el servidor.
- Definir filtros adecuados (por duración, base de datos y tipo de evento).
- Guardar en archivo y rotar periódicamente.
- Detener la sesión cuando termine el monitoreo.
- Analizar los resultados y aplicar acciones de optimización.

2 PROYECTO 2: IMPLEMENTACIÓN DE BACKUPS DIFERENCIALES PARA TRÁMITES ACADÉMICOS



ENUNCIADO

Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones.

La Dirección de Informática solicita construir una solución de monitoreo que capture automáticamente:

- Consultas lentas
- Deadlocks
- Bloqueos.
- Errores superiores al nivel 16.
- Uso excesivo de CPU.

Toda la información deberá almacenarse en un archivo de eventos para su posterior análisis.

SOLUCIÓN (T-SQL)

```
CREATE EVENT SESSION AuditoriaSistema
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
  ACTION(sqlserver.sql_text)
  WHERE duration > 1000000 -- consultas lentas
),
ADD EVENT sqlserver.lock_deadlock, -- deadlocks
ADD EVENT sqlserver.lock_acquired -- bloqueos
(
  WHERE duration > 5000
),
ADD EVENT sqlserver.error_reported -- errores > nivel 16
(
  WHERE severity >= 16
),
ADD EVENT sqlserver.rpc_completed -- alto CPU
(
  WHERE cpu_time > 1000
),
ADD TARGET package0.event_file
(
  SET filename = 'C:\Eventos\Auditoria.xel',
  max_file_size = 50,
  max_rollover_files = 10
);

ALTER EVENT SESSION AuditoriaSistema
ON SERVER STATE = START;
```



JUSTIFICACIÓN TÉCNICA

La sesión captura los eventos críticos que afectan el rendimiento del sistema y los almacena de forma automática.

Esto permite detectar problemas como bloqueos, deadlocks, errores graves y alto consumo de CPU sin intervención manual.



BUENAS PRÁCTICAS

- Monitorear de forma continua los eventos críticos.
- Revisar periódicamente los deadlocks y bloqueos.
- Analizar las consultas con mayor consumo de CPU.
- Rotar y limpiar los archivos de eventos para evitar llenar disco.
- Corregir los errores críticos a tiempo.

3 PROYECTO 3: ESTRATEGIA INTEGRAL FULL + DIFFERENTIAL + LOG BACKUP



ENUNCIADO

La universidad desea implementar un sistema permanente de diagnóstico del rendimiento para la base de datos GradosTítulos.

El sistema deberá:

- ✓ Registrar automáticamente las consultas más costosas.
- ✓ Almacenar la información en una tabla histórica.
- ✓ Permitir auditorías posteriores.
- ✓ Generar reportes de:
 - Mayor duración.
 - Mayor consumo de CPU.
 - Mayor número de lecturas lógicas.

SOLUCIÓN (T-SQL)

1 Crear tabla histórica

```
CREATE TABLE HistorialConsultas (
  Id INT IDENTITY(1,1) PRIMARY KEY,
  Fecha DATETIME DEFAULT GETDATE(),
  Consulta NVARCHAR(MAX),
  Duracion BIGINT, -- microsegundos
  CPU BIGINT, -- microsegundos
  Lecturas BIGINT -- lecturas lógicas
);
```

2 Crear sesión Extended Events

```
CREATE EVENT SESSION AuditoriaConsultas
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
ADD TARGET package0.event_file
(
  SET filename = 'C:\Eventos\Historico.xel',
  max_file_size = 50,
  max_rollover_files = 10
);

ALTER EVENT SESSION AuditoriaConsultas
ON SERVER STATE = START;
```

3 Importar eventos a la tabla

```
INSERT INTO HistorialConsultas
(Consulta, Duracion, CPU, Lecturas)
SELECT
event_data.value('event/action[\"name=sql_text\"]/value[1]', 'nvarchar(max)'),
event_data.value('event/data[@name=duration]/value[1]', 'bigint'),
event_data.value('event/data[@name=cpu_time]/value[1]', 'bigint'),
event_data.value('event/data[@name=logical_reads]/value[1]', 'bigint')
FROM EventosImportados;

(EventosImportados: resultado de leer el archivo .xel con sys.fn_xe_file_target_read_file)
```

4 Reporte: consultas más costosas

```
SELECT TOP 10
Fecha,
Duracion,
CPU,
Lecturas,
Consulta
FROM HistorialConsultas
ORDER BY Duracion DESC;
```



JUSTIFICACIÓN TÉCNICA

Este sistema permite mantener un historial permanente de las consultas ejecutadas, identificando las que generan mayor impacto en el servidor. Con los reportes, se pueden tomar decisiones para optimizar índices, consultas y recursos.



BUENAS PRÁCTICAS

- Utilizar Extended Events para menor impacto.
- Almacenar y analizar el historial de forma periódica.
- Revisar consultas con mayor duración, CPU y lecturas lógicas.
- Actualizar índices y estadísticas con regularidad.
- Eliminar archivos de eventos antiguos para liberar espacio.



CONCLUSIÓN

El uso de herramientas como SQL Profiler y Extended Events permite monitorear y analizar el rendimiento del sistema, detectar problemas de forma proactiva y aplicar estrategias de optimización que garanticen la disponibilidad, estabilidad y eficiencia de la base de datos GradosTítulos.



BASE DE DATOS II – TEMAS 2 Y 3

RESUMEN Y SOLUCIÓN DE PROYECTOS



RESUMEN GENERAL

TEMA 2. Estadísticas e índices

Objetivo: Optimizar el rendimiento de la base de datos mediante la correcta administración de índices y estadísticas.



- Analizar índices existentes.
- Detectar índices faltantes.
- Crear índices Clustered y NonClustered.
- Crear índices INCLUDE.
- Detectar fragmentación.
- Reorganizar o reconstruir índices.
- Actualizar estadísticas.
- Automatizar el mantenimiento de índices.

TEMA 3. Administración de transacciones y bloqueos

Objetivo: Garantizar la integridad de los datos cuando varios usuarios trabajan al mismo tiempo.

- Uso de transacciones.
- COMMIT y ROLLBACK.
- Manejo de errores con TRY...CATCH.
- Bloqueos (Locks).
- Deadlocks.
- Uso de DMV para monitoreo.
- Prevención de conflictos entre usuarios.



TEMA 2: ESTADÍSTICAS E ÍNDEXES

1 PROYECTO 1: OPTIMIZACIÓN DE ÍNDEXES PARA CONSULTAS DE ESTUDIANTES

Enunciado

Optimizar las consultas sobre las tablas:

- AcademicoPersona
- AcademicoMatricula
- Tramite
- TramitePersona

Justificación

Los índices reducen el tiempo de búsqueda y mejoran considerablemente el rendimiento de las consultas.



Buenas prácticas

- Crear índices solo en columnas consultadas frecuentemente.
- Evitar índices innecesarios.
- Actualizar estadísticas.

Solución (T-SQL)

```
-- Buscar índices faltantes
SELECT *
FROM sys.dm_db_missing_index_details;
```

```
-- Crear índice Clustered
CREATE CLUSTERED INDEX IX_Matricula
ON AcademicoMatricula(IdMatricula);
```

```
-- Crear índice NonClustered
CREATE NONCLUSTERED INDEX IX_Estudiante
ON AcademicoPersona(ApellidoPaterno);
```

```
-- Crear índice INCLUDE
CREATE NONCLUSTERED INDEX IX_Busqueda
ON AcademicoPersona(ApellidoPaterno)
INCLUDE(Nombres, DNI);
```

2 PROYECTO 2: DETECCIÓN Y CORRECCIÓN DE FRAGMENTACIÓN

Enunciado

Detectar la fragmentación de índices y aplicar la acción correctiva adecuada.

Justificación

La fragmentación incrementa las lecturas del disco y reduce el rendimiento.



Buenas prácticas

- Reorganizar si la fragmentación está entre 5 % y 30 %.
- Reconstruir cuando supere el 30 %.
- Actualizar estadísticas después del mantenimiento.

Solución (T-SQL)

```
-- Consultar fragmentación
SELECT *
FROM sys.dm_db_index_physical_stats
(DB_ID(), NULL, NULL, NULL, 'SAMPLED');
```

```
-- Reorganizar índices
ALTER INDEX ALL
ON AcademicoPersona
REORGANIZE;
```

```
-- Reconstruir índices
ALTER INDEX ALL
ON AcademicoPersona
REBUILD;
```

```
-- Actualizar estadísticas
UPDATE STATISTICS AcademicoPersona;
```



3 PROYECTO 3: AUTOMATIZACIÓN DEL MANTENIMIENTO DE ÍNDEXES

Enunciado

Automatizar el mantenimiento de índices y estadísticas para mejorar el rendimiento de forma periódica.

Justificación

Automatizar el mantenimiento evita degradaciones del rendimiento sin intervención manual.



Buenas prácticas

- Programar tareas periódicas.
- Registrar resultados.
- Revisar tiempos de ejecución.

Solución (T-SQL)

```
-- Actualizar estadísticas
EXEC sp_updatestats;
```

```
-- Reconstrucción de índices (si es necesario)
ALTER INDEX ALL
ON AcademicoPersona
REBUILD;
```

```
Ejemplo de creación de Job (SQL Server Agent)
EXEC sp_add_job @job_name = 'MantenimientoIndices';
EXEC sp_add_jobstep @job_name = 'MantenimientoIndices',
    @step_name = 'ActualizarIndices',
    @subsystem = 'TSQL',
    @command = 'EXEC sp_updatestats;
ALTER INDEX ALL ON AcademicoPersona REBUILD;';
EXEC sp_add_schedule @schedule_name = 'Diario 02:00',
    @freq_type = 4, @freq_interval = 1,
    @active_start_time = 020000;
EXEC sp_attach_schedule @job_name = 'MantenimientoIndices',
    @schedule_name = 'Diario 02:00';
EXEC sp_add_jobserver @job_name = 'MantenimientoIndices';
```



TEMA 3: ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

1 PROYECTO 1: CONTROL DE TRANSACCIONES EN LA APROBACIÓN DE TÍTULOS

Enunciado

Durante la aprobación de un título profesional intervienen varias tablas. Desarrollar una transacción que garantice:

- Atomicidad.
- Confirmación con COMMIT.
- Reversión con ROLLBACK.
- Manejo de errores con TRY...CATCH.
- Registro del error.

Justificación

Garantiza que todas las operaciones se completen correctamente o ninguna sea aplicada.



Buenas prácticas

- Siempre utilizar TRY...CATCH.
- Confirmar con COMMIT.
- Revertir con ROLLBACK.

Solución (T-SQL)

```
BEGIN TRY
    BEGIN TRANSACTION
    -- Actualizar trámite
    UPDATE Tramite
    SET Estado = 'Aprobado'
    WHERE IdTramite = 1;

    -- Registrar resolución
    UPDATE Resolucion
    SET Fecha = GETDATE()
    WHERE IdTramite = 1;
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    INSERT INTO LogErrores(Mensaje, Fecha)
    VALUES (ERROR_MESSAGE(), GETDATE());
END CATCH;
```

2 PROYECTO 2: SIMULACIÓN Y RESOLUCIÓN DE BLOQUEOS

Enunciado

Los coordinadores modifican simultáneamente el mismo trámite provocando bloqueos prolongados. Desarrollar un reporte que permita:

- Simular bloqueos.
- Detectarlos mediante DMV.
- Identificar la sesión bloqueadora.
- Liberar recursos correctamente.
- Elaborar un reporte del incidente.

Justificación

Permite identificar qué sesión está bloqueando otra y resolver el conflicto de forma adecuada.



Buenas prácticas

- Mantener transacciones cortas.
- Liberar recursos rápidamente.
- Evitar bloqueos innecesarios.



Solución (T-SQL)

```
-- Consultar bloqueos
SELECT * FROM sys.dm_tran_locks;

-- Consultar sesiones
SELECT * FROM sys.dm_exec_sessions;

-- Consultar solicitudes activas
SELECT * FROM sys.dm_exec_requests;

-- Finalizar sesión bloqueadora (ejemplo)
KILL <session_id>;
```

3 PROYECTO 3: PREVENCIÓN DE DEADLOCKS

Enunciado

Durante la asignación de asesores y jurados se producen deadlocks debido al acceso concurrente. Desarrollar un laboratorio que:

- Reproduzca un deadlock.
- Capture el gráfico XML.
- Analice la causa.
- Rediseñe el orden de acceso a tablas.
- Demuestre la eliminación del conflicto.

Justificación

Los deadlocks afectan la disponibilidad del sistema y deben identificarse para rediseñar el acceso concurrente.



Buenas prácticas

- Acceder a las tablas siempre en el mismo orden.
- Crear índices adecuados.
- Mantener transacciones lo más cortas posible.
- Evitar mantener bloqueos por mucho tiempo.



- ✓ SQL Profiler
- ✓ Extended Events
- ✓ DMV (sys.dm_*)
- ✓ SQL Server Agent



CONCLUSIÓN GENERAL

El uso adecuado de índices, estadísticas, transacciones, bloqueos y herramientas de monitoreo permite optimizar el rendimiento, garantizar la integridad de los datos y asegurar que múltiples usuarios trabajen simultáneamente sin afectar el desempeño del sistema.



BASE DE DATOS II

TEMAS 4 Y 5: ANÁLISIS DE PLANES DE EJECUCIÓN Y OPTIMIZACIÓN DE CONSULTAS T-SQL



TEMA 4. ANÁLISIS DE PLANES DE EJECUCIÓN



OBJETIVO

Analizar los planes de ejecución para identificar operaciones costosas que afectan el rendimiento y optimizar las consultas e índices.



ASPECTOS PRINCIPALES

- ✓ Analizar el plan de ejecución.
- ✓ Identificar operadores costosos.
- ✓ Detectar: Table Scan, Index Scan, Nested Loops, Hash Match, Sort.
- ✓ Comparar planes antes y después de crear índices.
- ✓ Optimizar procedimientos almacenados.

1 PROYECTO 1: DIAGNÓSTICO DE CONSULTAS COMPLEJAS



ENUNCIADO

Las consultas institucionales tardan más de 10 segundos. Se debe analizar el plan de ejecución para detectar operadores costosos y optimizar la consulta.

Qué buscar en el plan

- ⚠ Table Scan
- ⚠ Index Scan
- ⚠ Nested Loops
- ⚠ Hash Match
- ⚠ Sort
- ⚠ Costos altos

BUENAS PRÁCTICAS

- Revisar el costo estimado de cada operador.
- Evitar Table Scan mediante índices.
- Filtrar datos antes de realizar JOIN.

SOLUCIÓN (T-SQL)

```
-- Activar el plan de ejecución
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
```

```
-- Consulta de ejemplo
SELECT *
FROM AcademicoPersona A
INNER JOIN Tramite T
ON A.IdPersona = T.IdPersona;
```

JUSTIFICACIÓN

El plan de ejecución permite identificar operaciones costosas (Table Scan, Hash Match, Sort) y optimizar la consulta para mejorar el rendimiento.



TEMA 5. OPTIMIZACIÓN DE CONSULTAS T-SQL



OBJETIVO

Mejorar el rendimiento de las consultas y procedimientos, reduciendo el tiempo de ejecución y el consumo de CPU y recursos.



ASPECTOS PRINCIPALES

- ✓ Eliminar subconsultas innecesarias.
- ✓ Optimizar JOIN.
- ✓ Usar EXISTS.
- ✓ Usar CTE.
- ✓ Aplicar índices adecuados.
- ✓ Filtrar datos temprano.
- ✓ Parametrizar procedimientos.
- ✓ Evitar cursores innecesarios.
- ✓ Revisar Parameter Sniffing.
- ✓ Comparar tiempos y recursos.



1 PROYECTO 1: REESCRITURA DE CONSULTAS DE ALTA DEMANDA



ENUNCIADO

Optimizar el historial completo tes trámite de un estudiante, que consume etiempo de recursos.

CONSULTA NO OPTIMIZADA

```
SELECT *
FROM AcademicoPersona
WHERE IdPersona IN
(
    SELECT IdPersona
    FROM Tramite
);
```

CONSULTA OPTIMIZADA

```
SELECT A.*
FROM AcademicoPersona A
WHERE EXISTS
(
    SELECT 1
    FROM Tramite T
    WHERE T.IdPersona = A.IdPersona
);
```



JUSTIFICACIÓN

El uso de EXISTS evita recorrer registros innecesarios y mejora el rendimiento en grandes volúmenes de datos.

BUENAS PRÁCTICAS

- Reemplazar IN por EXISTS cuando sea conveniente.
- Evitar subconsultas innecesarias.
- Seleccionar solo las columnas requeridas.



2 PROYECTO 2: COMPARACIÓN DE PLANES ANTES Y DESPUÉS DE CREAR ÍNDICES



ENUNCIADO

Comprobar el impacto real de nuevos índices sobre las consultas del módulo de grados.

El estudiante deberá:

- Obtener el plan inicial.
- Crear índices adecuados.
- Obtener el nuevo plan.
- Comparar operadores.
- Comparar costo estimado.
- Comparar tiempo total.



JUSTIFICACIÓN

La comparación demuestra la reducción de lecturas, de operaciones y del tiempo de respuesta al utilizar índices adecuados.

SOLUCIÓN (T-SQL)

```
-- Crear índice para acelerar búsquedas
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON AcademicoPersona(DNI);
GO
```

```
-- Ejecutar la consulta nuevamente
```

```
SELECT *
FROM AcademicoPersona
WHERE DNI = '12345678';
```



BUENAS PRÁCTICAS

- Comparar siempre el plan antes y después.
- Crear índices solo cuando sean necesarios.
- Evitar índices duplicados o innecesarios.

2 PROYECTO 2: OPTIMIZACIÓN DE REPORTES ACADÉMICOS



ENUNCIADO

Los reportes institucionales integran información de más de ocho tablas. Se requiere optimizar su rendimiento.

SOLUCIÓN (T-SQL)

```
-- Uso de CTE para mejorar legibilidad y rendimiento
WITH Reporte AS
(
    SELECT T.IdPersona,
    COUNT(*) AS TotalTramites
    FROM Tramite T
    WHERE T.Estado = 'Aprobado'
    GROUP BY T.IdPersona
)
SELECT *
FROM Reporte
ORDER BY TotalTramites DESC;
```



JUSTIFICACIÓN

Las CTE permiten estructurar consultas complejas de forma más eficiente y facilitan la optimización del plan.

BUENAS PRÁCTICAS

- Aplicar filtros tempranos.
- Mantener estadísticas actualizadas.
- Crear índices adecuados en columnas usadas en JOIN y WHERE.



3 PROYECTO 3: OPTIMIZACIÓN DE PROCEDIMIENTOS ALMACENADOS



ENUNCIADO

Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento.

El estudiante deberá:

- Detectar operadores costosos.
- Eliminar Table Scan.
- Reducir lecturas lógicas.
- Mejorar reutilización del plan.
- Documentar la optimización obtenida.

SOLUCIÓN (T-SQL)

```
CREATE PROCEDURE BuscarPersona
@DNI VARCHAR(8)
AS
BEGIN
    SELECT IdPersona, Nombres, Apellidos
    FROM AcademicoPersona
    WHERE DNI = @DNI;
END;
GO
```



JUSTIFICACIÓN

La parametrización mejora la reutilización del plan, reduce compilaciones y disminuye el consumo de CPU.

BUENAS PRÁCTICAS

- Parametrizar consultas.
- Evitar SELECT *.
- Crear índices sobre columnas utilizadas en filtros.

3 PROYECTO 3: OPTIMIZACIÓN DE PROCEDIMIENTOS DE BÚSQUEDA



ENUNCIADO

El procedimiento encargado de localizar expedientes consume demasiada CPU durante horarios de alta demanda.

SOLUCIÓN (T-SQL)

```
CREATE PROCEDURE BuscarExpediente
@Codigo INT
AS
BEGIN
    SELECT E.IdExpediente, E.Codigo, E.Fecha, E.Estado
    FROM Expediente E
    WHERE E.Codigo = @Codigo;
END;
GO
```



JUSTIFICACIÓN

La parametrización reduce compilaciones repetidas, mejora el uso del plan en la caché y disminuye el consumo de CPU.

BUENAS PRÁCTICAS

- Evitar cursores innecesarios.
- Revisar problemas de Parameter Sniffing.
- Comparar el consumo de CPU y tiempo antes y después de optimizar.
- Usar SET STATISTICS TIME/IO para medir.



HERRAMIENTAS CLAVE



Execution Plan (Ctrl + M)
Visualiza el plan de ejecución.



SET STATISTICS IO, TIME
Mide lecturas lógicas y tiempo.



DMV's
sys.dm_exec_query_stats,
sys.dm_exec_requests,
sys.dm_db_index_usage_stats.

BENEFICIOS

- ✓ Consultas más rápidas y eficientes.
- ✓ Reducción del consumo de CPU y recursos.
- ✓ Menos bloqueos y mayor concurrencia.
- ✓ Mejor escalabilidad del sistema.
- ✓ Mejor experiencia para los usuarios.



RECOMENDACIONES GENERALES

- ★ Analizar antes de optimizar.
- ★ Crear índices adecuados y necesarios.
- ★ Mantener estadísticas actualizadas.
- ★ Parametrizar consultas y procedimientos.
- ★ Monitorear constantemente el rendimiento.



CONCLUSIÓN

El análisis del plan de ejecución y la optimización de consultas T-SQL permiten mejorar significativamente el rendimiento de la base de datos, garantizando tiempos de respuesta mínimos y un uso eficiente de los recursos del servidor.





TEMA 6: CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Administra y controla el uso de CPU y memoria en SQL Server para garantizar rendimiento y priorizar procesos importantes.



¿QUÉ ES RESOURCE GOVERNOR?

Es una característica de SQL Server que permite administrar los recursos del sistema (CPU y memoria) asignándolos de forma controlada a diferentes grupos de trabajo (Workload Groups) dentro de grupos de recursos (Resource Pools).



ASPECTOS CLAVE

- Crear Resource Pools para distribuir recursos (CPU y memoria).
- Crear Workload Groups para clasificar usuarios y aplicaciones.
- Clasificar conexiones mediante una función clasificadora.
- Limitar el uso de CPU y memoria para evitar saturación.
- Priorizar procesos críticos sobre procesos generales.
- Monitorear el consumo mediante DMV para tomar decisiones.

PROYECTOS Y SOLUCIONES

1 PROYECTO 1: ADMINISTRACIÓN DE CARGA POR TIPO DE USUARIO



ENUNCIADO

La universidad desea limitar el consumo de recursos de los usuarios administrativos sin afectar las consultas institucionales críticas. Configurar Resource Governor para:

- Crear Resource Pool
- Crear Workload Group
- Clasificar conexiones
- Limitar CPU
- Limitar memoria
- Verificar el funcionamiento

SOLUCIÓN (T-SQL)

1 Crear Resource Pool

```
CREATE RESOURCE POOL PoolAdministrativo
WITH (
    MAX_CPU_PERCENT = 30,
    MAX_MEMORY_PERCENT = 30
);
GO
```



2 Crear Workload Group

```
CREATE WORKLOAD GROUP GrupoAdministrativo
USING PoolAdministrativo;
GO
```



3 Aplicar configuración

```
ALTER RESOURCE GOVERNOR
RECONFIGURE;
GO
```



JUSTIFICACIÓN

Esta configuración limita el uso de CPU y memoria para los usuarios administrativos, evitando que afecten los procesos académicos más importantes.

BUENAS PRÁCTICAS

- Crear grupos según el tipo de usuario.
- No asignar el 100 % de CPU a un solo grupo.
- Supervisar periódicamente el consumo.

2 PROYECTO 2: PRIORIZACIÓN DE PROCESOS CRÍTICOS



ENUNCIADO

Durante el cierre del semestre, la emisión de diplomas debe tener mayor prioridad que las consultas normales.

SOLUCIÓN (T-SQL)

1 Crear Resource Pool de alta prioridad

```
CREATE RESOURCE POOL PoolCritico
WITH (
    MAX_CPU_PERCENT = 80,
    MAX_MEMORY_PERCENT = 70
);
GO
```



2 Crear Workload Group

```
CREATE WORKLOAD GROUP GrupoCritico
USING PoolCritico;
GO
```



3 Aplicar cambios

```
ALTER RESOURCE GOVERNOR
RECONFIGURE;
GO
```



JUSTIFICACIÓN

Los procesos críticos disponen de más recursos para ejecutarse rápidamente durante periodos de alta demanda.

BUENAS PRÁCTICAS

- Priorizar únicamente procesos esenciales.
- Monitorear el impacto sobre otros usuarios.
- Ajustar los porcentajes según la carga del servidor.

3 PROYECTO 3: MONITOREO DEL CONSUMO DE RECURSOS



ENUNCIADO

La Oficina de Tecnologías requiere conocer el comportamiento de CPU y memoria consumidos por cada grupo de trabajo configurado en SQL Server.

SOLUCIÓN (T-SQL) – CONSULTAS DMV

1 Consultar Resource Pools

```
SELECT *
FROM sys.dm_resource_governor_resource_pools;
```



2 Consultar Workload Groups

```
SELECT *
FROM sys.dm_resource_governor_workload_groups;
```



3 Consultar solicitudes activas

```
SELECT *
FROM sys.dm_exec_requests;
```



4 Consultar sesiones activas

```
SELECT *
FROM sys.dm_exec_sessions;
```



JUSTIFICACIÓN

Las DMV permiten supervisar en tiempo real el uso de CPU, memoria, sesiones activas y grupos de trabajo, facilitando la toma de decisiones para optimizar el rendimiento.

BUENAS PRÁCTICAS

- Supervisar continuamente el consumo de recursos.
- Identificar consultas con alto consumo de CPU.
- Revisar sesiones inactivas o bloqueadas.
- Ajustar los Resource Pools según la demanda.
- Generar reportes periódicos para evaluar el comportamiento del servidor.

¿CÓMO FUNCIONA?



BENEFICIOS

- ✓ Mejor control del uso de CPU y memoria.
- ✓ Evita que cargas pesadas afecten a otros usuarios.
- ✓ Prioriza procesos críticos del negocio.
- ✓ Mejora la estabilidad y disponibilidad del servidor.
- ✓ Optimiza el rendimiento general del sistema.



EJEMPLO DE CLASIFICACIÓN (FUNCIÓN)

```
CREATE FUNCTION dbo.fn_ClasificarUsuario()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @grupo sysname;
    IF IS_MEMBER('db_admino') = 1
        SET @grupo = 'GrupoAdministrativo';
    ELSE
        SET @grupo = 'GrupoInstitucional';
    RETURN @grupo;
END;
GO
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = dbo.fn_ClasificarUsuario);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
```



CONCLUSIÓN

Resource Governor permite administrar los recursos de SQL Server de manera inteligente, asegurando que las cargas importantes tengan prioridad y que el servidor funcione de forma eficiente, estable y controlada.



¡Un buen control de recursos hoy, garantiza un mejor rendimiento mañana!